



南開大學
Nankai University

计算机学院
软件工程实验报告

实验 2: 智能城市生成系统——需求分析报告

姓名：王旭

学号：2312166

专业：计算机科学与技术

2026 年 4 月 18 日

目录

1	引言	3
1.1	编写目的	3
1.2	项目背景	3
2	任务概述	4
2.1	任务目标	4
2.2	用户特点	4
2.3	假定与约束	5
2.3.1	假定条件	5
2.3.2	约束条件	6
3	业务描述	6
3.1	系统总业务流程图及其描述	6
3.1.1	系统总业务流程图	7
3.1.2	系统总业务流程描述	8
3.2	各个子业务流程图及其描述	8
3.2.1	多角色登录与系统初始化子业务	8
3.2.2	智能语言交互与仿真环境控制子业务（创新核心）	9
3.2.3	图像草图驱动与布局修改子业务	11
3.2.4	资产联动配置与替换子业务	12
4	数据需求	13
4.1	数据需求描述	13
4.2	数据流图 (DFD)	14
4.2.1	顶层数据流图 (0 层 DFD)	14
4.2.2	第一层数据流图 (1 层 DFD)	15
4.3	数据字典	15
4.3.1	数据存储字典	15
4.3.2	核心数据流字典	17
5	功能需求	18
5.1	功能划分	18
5.2	功能描述	19
5.2.1	界面与架构管理模块	19
5.2.2	场景生成模块	20
5.2.3	布局与生态控制模块	21
5.2.4	智能交互与仿真模块	21
5.2.5	多模态指令解析模块（创新模块）	22
6	性能与非功能需求	22
6.1	准确性	22
6.2	及时性	23

6.3	可扩充性	23
6.4	易用性	23
6.5	易维护性	24
6.6	标准性与先进性	24
7	系统运行要求	24
7.1	硬件配置要求	24
7.2	软件配置要求	25
8	总结	26

智能城市生成系统——需求分析报告

1 引言

1.1 编写目的

本文档是智能城市生成系统的软件需求分析报告。编写本报告的主要目的是全面且清晰地说明系统的各项功能需求、业务流程、数据要求以及性能指标。本报告从用户的实际应用场景出发，将用户对场景建模自动化、智能化的期望，转化为具体的软件开发要求，为后续的设计与开发工作提供明确的指导。

具体来说，编写本文档旨在实现以下几个目标：

- 明确系统需求：**为项目的所有参与者（包括提出需求的用户、管理人员和开发团队）提供一份清晰的需求说明，确保大家对系统最终需要实现的功能有一致的理解，减少前期沟通中的误解，并作为后续版本迭代的参考依据。
- 指导后续开发：**作为系统架构设计、数据库结构设计以及具体功能代码编写（如 LLM 大模型指令解析、Blender 插件集成等）的主要参考标准。开发人员可以根据本文档来规划技术方案并开展编码工作。
- 提供测试与验收标准：**为软件测试人员提供详细的参考依据。后续的功能测试用例设计、性能测试以及最终的软件交付验收，都将以本文档中描述的内容为准。

根据本系统的特点，本文档的预期读者主要分为以下四类：

- 系统的最终用户：**即**场景建模师、行业分析师与系统管理员**。他们可以通过阅读本文档（尤其是功能描述部分），核对并确认系统是否满足了他们工作中的实际诉求，例如“2D/3D 资产替换”、“自然语言自动控制建模”等，并对最终的需求内容进行确认。
- 项目管理人员：**主要用于了解软件的整体边界和功能范围，方便评估整体的开发工作量、制定合理的开发时间表，并协助跟进项目进度，避免在开发过程中随意偏离原定的需求。
- 软件开发团队：**包括前后端开发人员、Blender 插件工程师以及大模型（LLM）开发工程师。开发人员将以本文档作为编写代码、设计接口和处理业务逻辑的直接说明书。
- 软件测试人员：**用于熟悉系统的各项功能要求和运行条件，从而编写测试用例、规划测试流程，确保最终交付的系统完全符合需求规定。

1.2 项目背景

随着城市化进程的不断加快以及数字孪生概念的普及，城市建模逐渐从单纯的视觉展示向智能化、数据化和交互性方向发展。传统的城市建模工作高度依赖人工操作，场景建模师往往需要花费大量时间在基础资产的摆放、环境生态的配置以及重复性修改上。这种工作模式不仅效率低下，而且难以快速响应复杂多变的设计需求。

近年来，人工智能技术（特别是大语言模型 LLM）与 3D 建模技术的深度融合，为城市建模领域带来了根本性的突破。现代智能生成系统开始广泛结合自然语言处理、图像识别和动态仿真等前沿技

术。用户可以通过更加直白和自然的方式（如一句话或一张草图）来表达设计意图，而系统则能自动将这些意图转化为具体的 3D 场景并进行渲染。这使得城市建模的自动化与智能化水平得到了显著提升。

基于上述背景，“智能城市生成系统”应运而生。本系统的开发旨在为场景建模师和行业分析师等相关从业者提供一个更加高效、精准和具象化的城市设计平台。一方面，系统通过整合丰富的 2D 与 3D 资产库以及参数化配置模板，大幅缩短了基础场景的搭建时间；另一方面，也是本系统最具前瞻性的意义所在，即通过接入大语言模型（LLM），系统能够实现多模态指令解析，允许用户以自然语言文本自动调用底层插件的生成函数。这意味着未来的场景建模可以使用类似对话沟通的智能工具来辅助完成，从而极大提高开发效率，赋予城市建模工具更高的“逻辑理解能力”与“智能交互水平”。

2 任务概述

2.1 任务目标

本系统的核心任务是开发一款深度融合自然语言处理、图像识别、动态仿真与 3D 建模技术（主要依托 Blender）的“智能城市生成系统”。该系统旨在通过引入人工智能（特别是大语言模型 LLM）等智能化工具，替代传统建模中繁琐的人工操作，为场景建模师和行业分析师提供一个高效、精准且具象化的城市设计平台。

为实现上述核心诉求，本项目的具体开发目标可分解为以下几个方面：

- 提升场景搭建效率：**实现系统与 Blender 插件系统的无缝交互。系统需要支持 2D 与 3D 资产的快速加载和替换，并能够通过模板选择轻松实现基础元素（如树木、道路、座椅等）的自动联动配置，大幅缩短基础场景的搭建时间。
- 实现多样化的布局与生态控制：**一方面，保证系统可以通过精确的输入数据（如坐标点集）来严格控制城市布局；另一方面，支持通过分析手绘的图像草图来提取特征点线信息，从而进行城市布局的修改。此外，系统还需具备在城市外围自动生成山峦、湖泊、游船等自然与生态元素的能力。
- 打造自然语言智能交互体验（创新目标）：**深度接入大语言模型（LLM），赋予系统多模态指令解析能力。系统需要能够准确理解用户的自然语言输入，并自动将其转换为对应的插件生成函数来执行建模操作。系统还支持用户通过文字直接控制天色、天气等环境参数。
- 引入符合真实规则的动态仿真：**突破静态建模的限制，系统将支持在已生成的城市场景中加入动态元素，例如按照真实社会交通规则运行的汽车轨迹、以及具备独立行进逻辑的人群模拟，极大地增强场景的生动性与真实感。
- 建立多层次的安全与管理架构：**开发一套完善的用户管理体系，支持多角色的注册、登录与权限分配。系统需要能够根据当前登录的角色（如场景建模师、行业分析师或系统管理员）自动显示相对应的功能界面（UI）与操作权限。

2.2 用户特点

本系统的最终用户主要分为三类：场景建模师、行业分析师和系统管理员。由于各角色的专业背景与工作侧重点不同，系统在权限分配、UI 界面与功能入口上将为他们提供差异化的支持。具体的用户特点分析如下：

1. 场景建模师

- **角色职责**：城市 3D 场景的主要创建者，把控呈现出的视觉效果。
- **技术水平**：精通 3D 空间结构与建模理论，熟悉主流建模软件（尤其是 Blender）的各项复杂操作，但通常不具备深厚的软件开发或代码编写能力。
- **业务诉求**：他们期望系统能够极大减少“摆放树木、铺设道路、摆放座椅”等繁杂的重复性体力劳动。主要诉求包括：需要顺畅的主系统与 Blender 插件交互入口；便捷的 2D/3D 资产快速加载与替换；通过手绘草图提取点线集或手动输入坐标点集来精确修改布局；以及能够利用自然语言直接调用插件生成函数，从而将有限的时间和精力投入到场景的艺术表现和核心设计中。

2. 行业分析师

- **角色职责**：利用生成的城市 3D 场景进行数据分析、方案推演和业务评估（如城市规划验证、交通流调度仿真等）。
- **技术水平**：具备丰富的行业领域知识（如城市交通学、城市规划等），但对底层的专业 3D 建模操作（如顶点编辑、面片拉伸等）通常不熟悉。
- **业务诉求**：追求“所说即所得”的极简操作体验，不需要看到复杂的纯技术建模面板。他们高度依赖系统针对其角色展示的低门槛 UI 界面，需要利用大模型（LLM）的多模态指令解析能力——仅通过输入自然语言文本，即可控制天色/天气等环境参数，或向场景中一键添加符合社会规则运行轨迹的汽车与人群，以此直观地进行动态模拟与分析。

3. 系统管理员

- **角色职责**：负责系统的日常运维、数字资产更新以及平台用户的统筹管理。
- **技术水平**：具备专业的 IT 技术背景，熟悉系统网络架构、数据库管理、接口配置及网络安全维护。
- **业务诉求**：需要一个稳定且功能全面的运行管理后台。主要诉求包括：多角色（建模师、分析师等）账户的配置、登录管理与权限分配；系统中海量 2D 与 3D 资产的统一上传、分类及版本维护；以及监控大模型（LLM）接口的运转状态和调用情况，确保整个智能生成平台的高效且安全运转。

2.3 假定与约束

在系统开发与后续运行的过程中，我们需要明确以下假定与约束条件，这些条件划定了系统的技术边界和实施基础：

2.3.1 假定条件

1. **底层建模环境假定**：假定系统的 3D 渲染与核心场景生成功能强依赖于 Blender 软件生态。系统的大部分自动化操作（如资产加载替换、生态环境生成、仿真动画应用）均需要通过执行 Blender 插件脚本或调用其 Python API (bpy) 来实现。
2. **大语言模型 (LLM) 可用性**：假定项目在开发和运行期间，能够持续且稳定地接入外部或本地部署的大模型 API。系统所主打的多模态指令解析、自然语言交互（如利用文字切换天气、控制建模）均建立在 LLM 接口稳定且具备较强推理能力的基础上。

3. **基础资产库完备性**：假定在系统投入实际使用前，系统管理员或相关团队已经准备并梳理好了一套标准化的 2D 与 3D 资产库（涵盖各类型的树木、道路模板、山峦、动态车辆等），以便系统在接收到控制指令时能够随时进行加载和实例化。

2.3.2 约束条件

1. **接口响应与网络延迟约束**：由于“自然语言解析为 3D 场景动作”需要与 LLM 服务进行数据交互，系统的整体响应速度将不可避免地受限于网络带宽和模型推理耗时。因此，系统在架构上必须约束并容忍一定的接口延迟（如复杂指令下可能存在数秒的处理等待），并且在 UI 层面必须配置友好的加载提示。
2. **软硬件配置约束**：城市级的 3D 场景静态渲染及车辆人群的动态模拟对计算机硬件（尤其是 GPU 显存和处理器性能）消耗极大。系统必须运行在达到特定硬件基准的工作站上；同时，不同用户角色的 UI 端（如分析师的轻量级 Web 界面）需要与运行 Blender 的主节点建立稳定且无限制的本地或内网通信端口。
3. **指令解析的边界约束**：虽然系统引入了大模型，但受限于当前的领域专业词汇与技术边界，系统的自然语言控制范畴将被约束在“城市设计、环境影响、基础资产调配”的专门语境中。对于超出该范围（如非建模类的闲聊或过于抽象的艺术描述），系统可能无法成功将其转化并映射为明确的插件函数调用。

3 业务描述

3.1 系统总业务流程图及其描述

本系统的总体业务流程涵盖了从用户身份验证、工作台初始化、多模态智能交互、到底层 Blender 场景生成的完整闭环。系统根据不同用户角色提供差异化的入口，但其核心的场景生成数据流是统一的。

3.1.1 系统总业务流程图

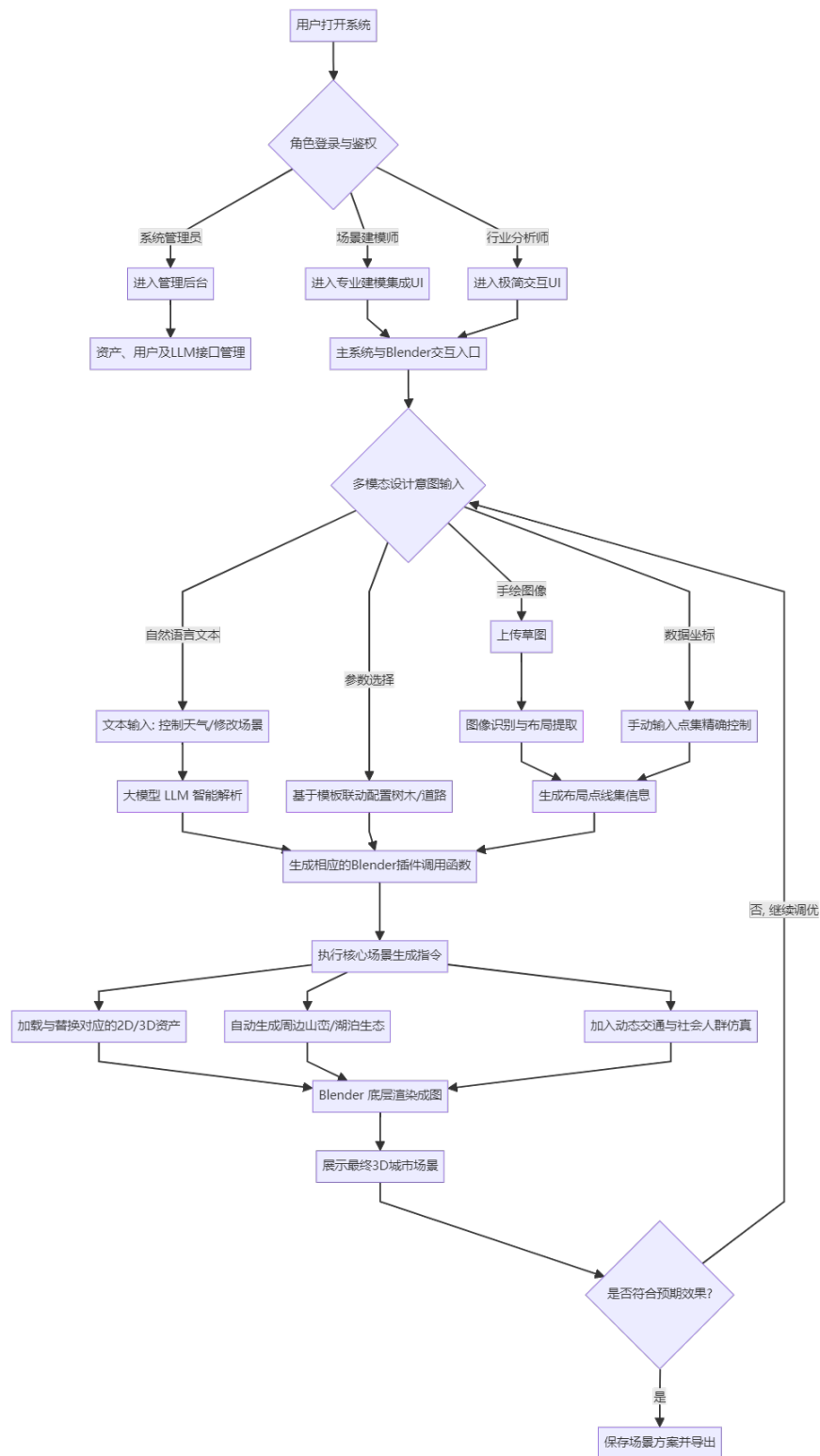


图 3.1: 系统总业务流程图

3.1.2 系统总业务流程描述

1. **用户登录与系统初始化**：用户启动系统后，首先进行身份验证与权限识别。系统根据角色（管理员、建模师、分析师）分配不同的用户界面。管理员进入后台执行资产与接口维护；建模师与分析师进入相应的工作台，建立主系统与 Blender 插件的底层通信链路。
2. **多模态意图输入**：用户通过各自的 UI 界面表达场景设计意图。系统支持四种并行的输入模式：
 - **文本输入**：利用自然语言下达指令（如“将天气切换为黄昏”、“在路边添加一圈树木”）。
 - **图像输入**：上传场景规划的手绘图像草图。
 - **数据输入**：精确地键入目标点集坐标数据以控制布局。
 - **模板配置**：通过下拉菜单设置道路、树木与座椅的联动组合参数。
3. **意图获取与智能解析**：系统进入核心的数据处理与解析阶段。自然语言文本指令被发送至内/外部的大语言模型（LLM）接口，经过理解与翻译后转化为可直接执行的 Blender 插件生成函数代码；图像草图则通过图像识别引擎提取出特征点线集数据，并与数据模式下的输入指令汇总为布局控制参数。
4. **场景生成与动态仿真**：各种解析完毕的插件调用函数和布局指令汇聚到系统的核心引擎。系统将从数据库中自动加载并替换对应的 2D/3D 基础资产（如建筑、树木模型），在主市区周边自然生成山峦、湖泊及游船等生态元素，最后将符合现实交通和社会规则的动态车辆与人群信息加入到城市街道中。
5. **渲染输出与反馈循环**：系统调用 Blender 完成城市场景的最终视觉渲染，并在屏幕上展示给用户。如果用户对生成效果不满意，可继续输入自然语言或坐标进行修改指令（即回到多模态输入环节）；如果满足需求，则流程结束，用户可进行工程文件保存或效果图导出。

3.2 各个子业务流程图及其描述

要实现对总业务的全面覆盖说明，我们要详细阐述核心的前端场景生成业务与系统的入口管理。因此，我们将总流程完整地拆解为**四个核心子业务**，并为每个子业务提供详细的步骤说明与异常处理描述。

3.2.1 多角色登录与系统初始化子业务

此流程涵盖了系统在使用前的入口控制，对应了多角色权限分配与界面分发，以及最重要的主系统与 Blender 插件交互通信的建立。

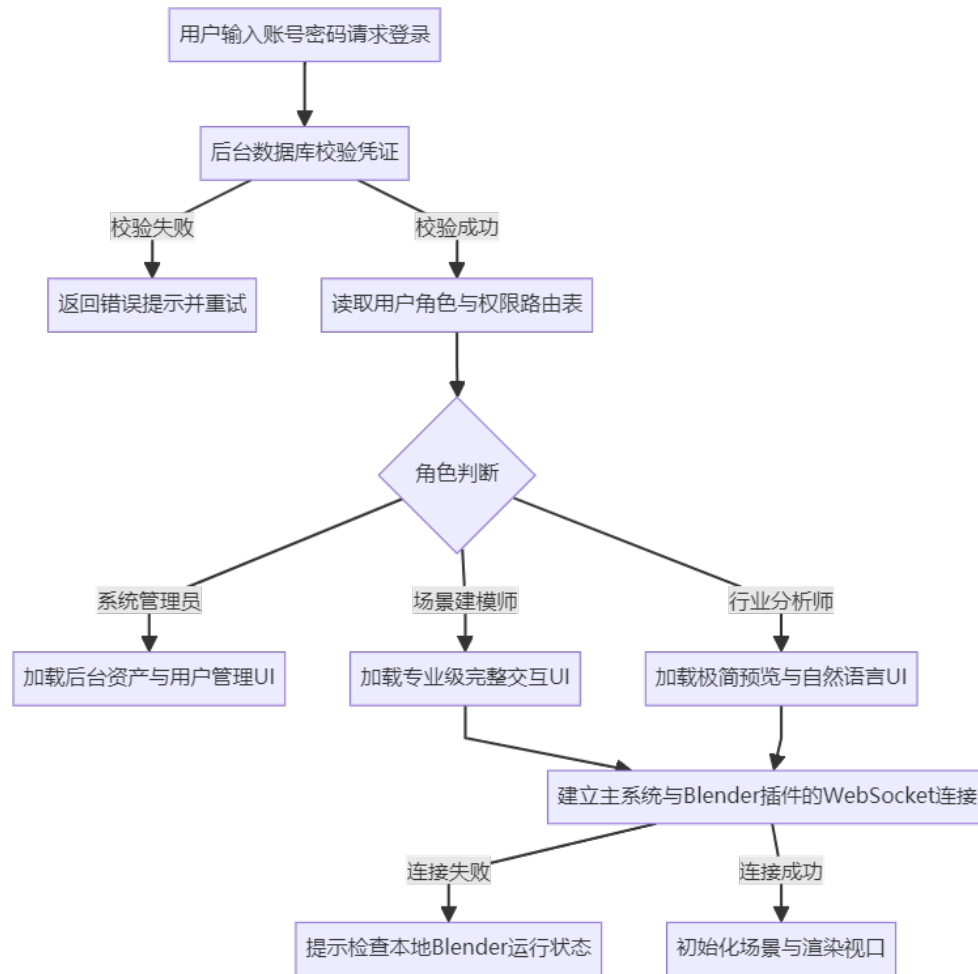


图 3.2: 多角色登录与系统初始化子业务

详细子业务描述：

- 身份凭证校验与信息获取：**用户在系统前端输入账号密码后，请求将发送至系统。若凭证有误，系统阻断访问；若成功，则会基于数据库返回该用户的身份令牌（Token）以及绑定的角色类型。
- 动态路由与 UI 渲染：**前端接收到角色类型后，执行动态路由拦截与组件挂载。系统管理员将直接被引导至纯数据的运维后台；场景建模师会看到包含画笔提取、资产模板选择等高级工具的专属界面；行业分析师的界面则会自动屏蔽底层参数，仅保留基于自然语言与模板的悬浮对话框以及场景视口。
- 建立底层跨进程通信（关键交互）：**对于需要预览 3D 场景的建模师和分析师，前端加载完毕后，主系统主程序会立即尝试通过 WebSocket 或本地端口（如 `localhost:8080`）与后台运行的 Blender 以及挂载的插件系统进行握手连接。如果未检测到 Blender 进程或插件未激活，则会明确提示用户检查系统环境；如果连接成功，则建立双向通信通道，完成初始场景渲染准备。

3.2.2 智能语言交互与仿真环境控制子业务（创新核心）

此流程描述了系统如何将用户的自然语言（如：“把天气变成下雨天，并在路上加上车辆”）转化为实际的 3D 仿真环境变化。

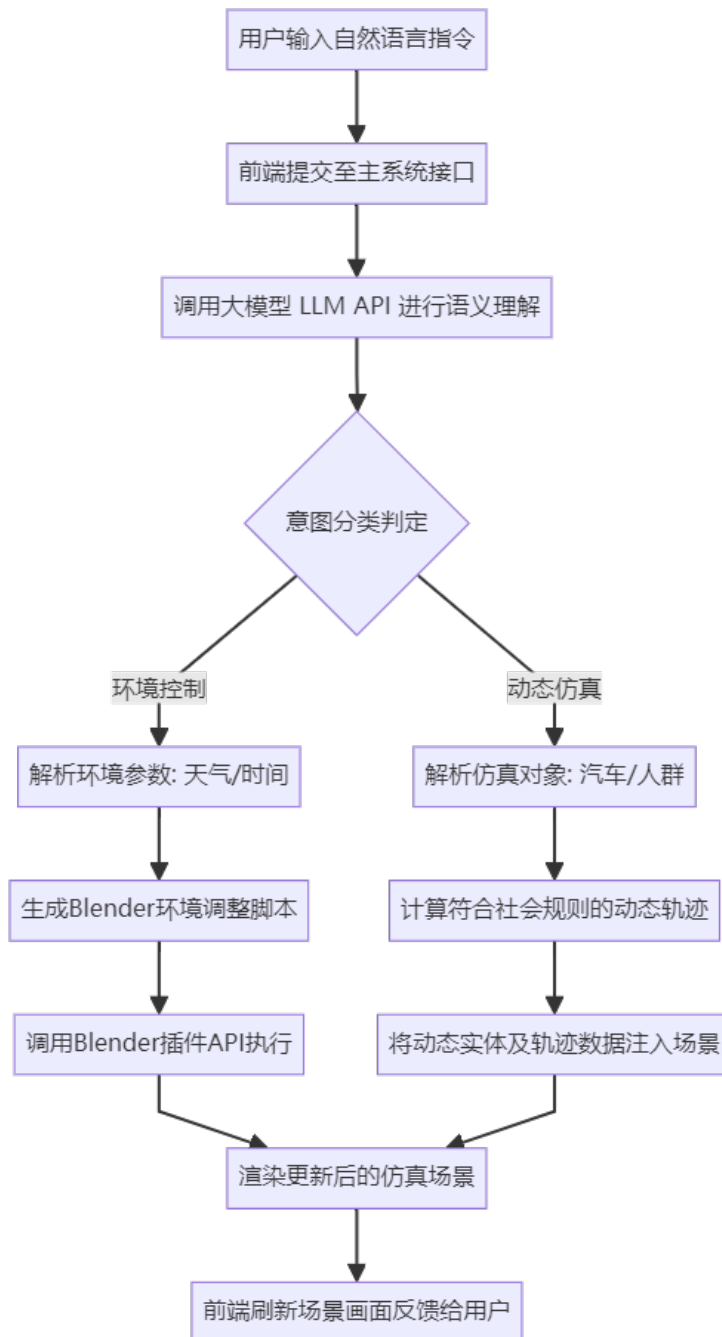


图 3.3: 智能语言交互与仿真环境控制子业务

详细子业务描述:

- 指令接收与 LLM 语义解析:** 用户在文本框内触发自然语言指令后, 主系统连同当前的场景状态上下文, 一并打包发送至大模型 (LLM) 的接口。LLM 通过内部的语义引擎解析用户的口语化表达, 判定这句话的主要意图。
- 环境参数解析与处理:** 若意图被识别为“环境控制”, 大模型会提取出具体的关键词 (如时间为“傍晚”, 天气为“暴雨”)。系统随之自动将其翻译为 Blender 端内的环境控制脚本, 包括调整世界环境光 (HDRI) 的角度与色温, 并激活动态的粒子系统 (雨雪特效)。
- 仿真对象解析与轨迹计算:** 若意图为“动态仿真” (如“生成穿梭在马路上的车流和人群”), 系

统不仅要在资产层面调用汽车与人体模型，更底层的逻辑是调用内置的模拟仿真算法。算法需要读取当前道路网的边界与人行道信息，生成一系列避免碰撞、遵循红绿灯等社会交通规则的运动路径集。

4. **引擎执行与回传**：无论是环境脚本还是携带物理轨迹的实体信息，最终均下发给 Blender 的插件系统执行。Blender 在后台渲染出最新的一帧（或实时动画）后，利用之前建立好的通信信道，将画面流实时推送到用户的前端界面中，完成一次完整的“所说即所得”的交互流程。

3.2.3 图像草图驱动与布局修改子业务

此流程描述了系统如何通过识别分析手绘草图并结合人工坐标参数，精准地修改或重构场景布局。

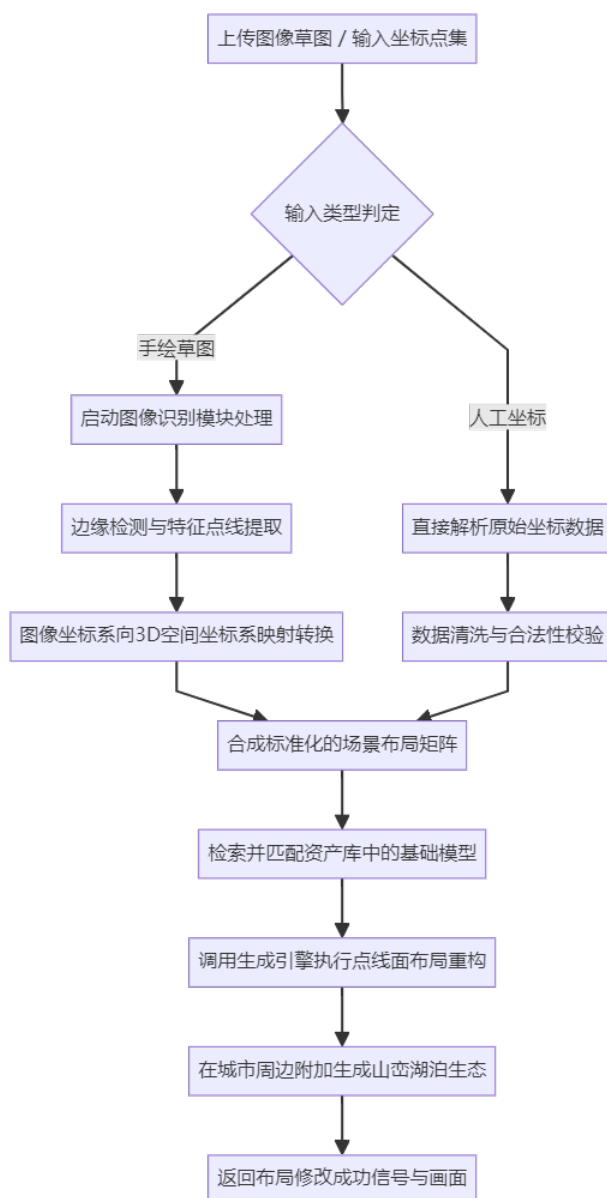


图 3.4: 图像草图驱动与布局修改子业务

详细子业务描述：

1. **多路径空间数据录入**：建模师在修改布局时，既可以直接在面板格系输入精准的长宽、间距坐标点集合，也支持直接上传一幅扫描的平面城市设计草稿图，以此覆盖不同的作业习惯。
2. **草图特征提取与坐标系映射**：针对图像草图，主系统需利用计算机视觉（CV）算法模块过滤杂色并检测图像边缘，识别出哪些线条代表主干道，哪些区块代表楼丛。提取出二维像素特征后，系统通过一个专门的换算器，将其缩放并映射转化为适配于真实 Blender 3D 世界的基础空间系网格坐标（通常为 X、Y 绝对坐标与预设的 Z 轴高度基准）。对于人工录入的坐标也是同理，须经历史数据格式一致性校验。
3. **布局矩阵合成与生态自生成**：被清洗和提取好的点、线数据会被组合成为一个“场景布局矩阵图”。Blender 插件监听到该矩阵变化后，执行底层布尔运算与实例化摆放，推翻原有路网构建出新的道路与建筑网格；同时，系统触发自发式的空间算法，识别城市的边界点，并在视线的尽头自动填补山峦、水池、游船等自然生态大背景。
4. **校验防碰撞并反馈结果**：在模型依据布局矩阵生成后，系统还需要做最基础的空间干涉检测，以免楼房穿模或道路断裂。完成后，新布局的界面同步呈现给用户。

3.2.4 资产联动配置与替换子业务

此部分聚焦于最基础也是使用最频繁的资产操作：即如何通过模板下拉框实现街道元素的智能联动以及 2D/3D 模型替换。

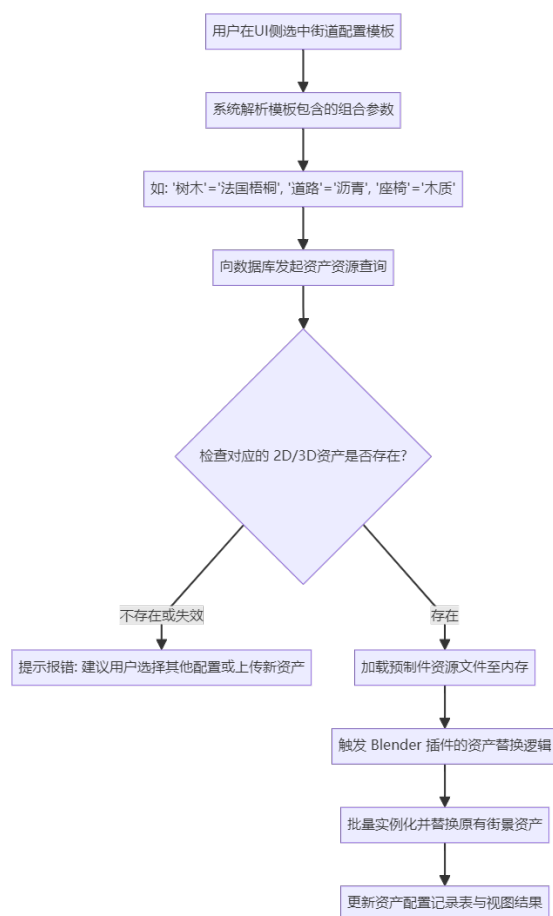


图 3.5: 资产联动配置与替换子业务

详细子业务描述：

1. **模板方案触发与参数拆解**：场景建模师从预先配置好的风格模板中选中某一项（例如“欧洲古典风商业街”模板）。主系统收到触发指令后，会在本地查表，将这个模板方案横向拆解为数个具体的资产要求：例如要求路灯变为“复古煤气灯 _ 资产 ID_002”，道路变为“石板路 _2D 材质_008”，树木变为“常绿灌木 _ 资产 ID_015”。这也这就是所谓的“联动配置”。
2. **资产库检索与状态校验**：拆解获得具体 ID 列表后，系统第一时间请求服务器的数字资产库，校验这批 2D 贴图与 3D 模型的物理文件是否存在、本地缓存是否已过期以及用户是否有权限调用。若缺少文件，立刻阻断流程并提醒用户补充下载；若正常，则将资产路径指针准备完毕。
3. **批量模型加载与替换运算**：得到有效指令与资源的 Blender 插件，会针对场景中原有的旧树木、老路面定位其原点位置和缩放比例；然后在内存中加载新的高精度预制模型和材质贴图。
4. **渲染缓存刷新**：贴图与模型更换完毕后，系统更新当前的场景方案树菜单，提示联动更改成功，屏幕光影也因材质参数的替换发生变化。

4 数据需求

4.1 数据需求描述

在“智能城市生成系统”的运行过程中，数据的输入、处理、存储与输出是支持系统各项业务正常开展的基础。有别于传统的管理信息系统（MIS），本系统在数据层面除了常见的字符串与关系型表单外，更广泛地涉及到了图像数据、非结构化文本以及大规模的 3D 模型资产文件。我们将系统的数据需求划分为以下四大类别：

1. 用户与配置管理数据（结构化数据）

- **用户凭证与权限数据**：记录场景建模师、行业分析师及系统管理员的账号信息（如 `User_ID`、密码 Hash）、以及他们所绑定的角色权限类型配置。
- **工程与模板方案数据**：记录用户存储的各版本城市设计工程信息（如 `Project_ID`、创建时间），以及系统中预设的用于道路、树木等元素的“联动置换模板参数表”。

2. 资产存储库数据（二进制与元数据结构）

- **资产物理文件数据**：这是系统生成场景的基础素材。包括高精度的 3D 模型文件（如 `.obj`，`.fbx`，`.blend` 格式的建筑、车辆、人员）、2D 纹理贴图（如 `.png`，`.jpg` 漫反射/法线/粗糙度贴图），以及相关的环境光照背景文件（如 `.hdr` 文件）。
- **资产元数据库**：是对上述物理文件的标签化描述。记录各类模型及材质的 `Asset_ID`、默认缩放比例、分类（如“植物 _ 乔木”、“交通工具 _ 轿车”）、上传时间以及碰撞边界盒（Bounding Box）参数。

3. 多模态交互与过程解析数据（半结构化与流数据）

- **自然语言请求与回调数据**：包括用户端发起的未经处理的口语化文字字符串（如“帮我加一排梧桐树”），以及经由大语言模型（LLM）API 回传的 `JSON` 结构的语义分析参数和自动生成的用于底层调用环境或模型的 Python 脚本程序。

- **图像与特征矩阵数据**：包含用户上传的 2D 城区手绘草稿图的位图数据矩阵，以及利用图像识别算法提取特征并进行空间转换后，生成的用来重构空间布局的【多边形面集】和【空间点坐标集(X,Y,Z)】。

4. 场景渲染与动态仿真数据（大规模空间数据）

- **城市静态框架与布局数据**：描述生成完成的主城区的建筑区块矩阵数据、路网节点分布以及周边生态元素（山峦、湖泊位置信息）。这些数据直接构成了 Blender 内部的基础节点信息。
- **动态仿真时序数据**：指系统计算并生成的包含车辆行驶、人员步行的动画关键帧轨迹矩阵集（如每一帧下独立实体的空间位置与状态），并在视口中连续渲染输出。

4.2 数据流图 (DFD)

数据流图 (Data Flow Diagram) 用于从数据传递和加工的角度，以图解的方式刻画系统的逻辑模型。为了清晰呈现智能城市生成系统中繁杂的多模态数据、资产数据与控制脚本的传递过程，我们将系统数据流划分为顶层数据流图 (0 层) 和第一层数据流图 (1 层)。

4.2.1 顶层数据流图 (0 层 DFD)

顶层数据流图将“智能城市生成系统”作为一个整体的处理黑盒，主要展示系统与其外部实体（即系统的三类目标用户）之间的顶层数据交互关系。

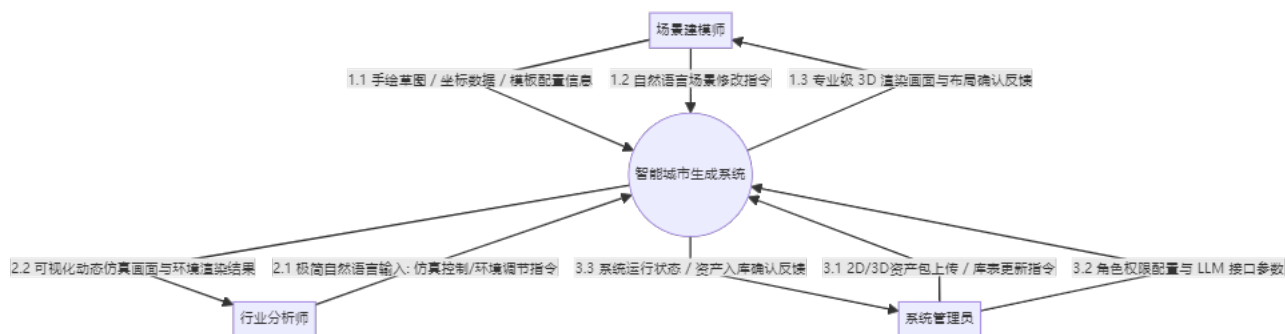


图 4.6: 顶层数据流图 (0 层 DFD)

顶层数据流描述：

- **场景建模师**向系统输入多元化的场景控制数据（如扫描的手绘草图矩阵、手动录入的精确空间坐标集、以及选择的模板联动参数），并可以通过自然语言下达修改指令；系统则向其输出高精度的 3D 渲染画面，反馈由数据转换而来的布局重构结果。
- **行业分析师**主要通过自然语言直接向系统输入环境控制参数（如天气、时间变更）和动态仿真命令（如加入车流人群）；系统解析后向其端侧输出直观的、附带社会物理规则运行的动态仿真场景画面。
- **系统管理员**负责向系统输入基础的维系数据，包括海量的 2D 贴图与 3D 预制件资产包格式文件，以及系统维度的 LLM 接口配置和人员权限表；系统返回资源入库的状态日志及运维监控图表。

4.2.2 第一层数据流图 (1 层 DFD)

第一层数据流图展示系统内部的核心处理模块 (Processes) 以及系统内部数据存储库 (Data Stores) 之间的数据传递和转换流向。

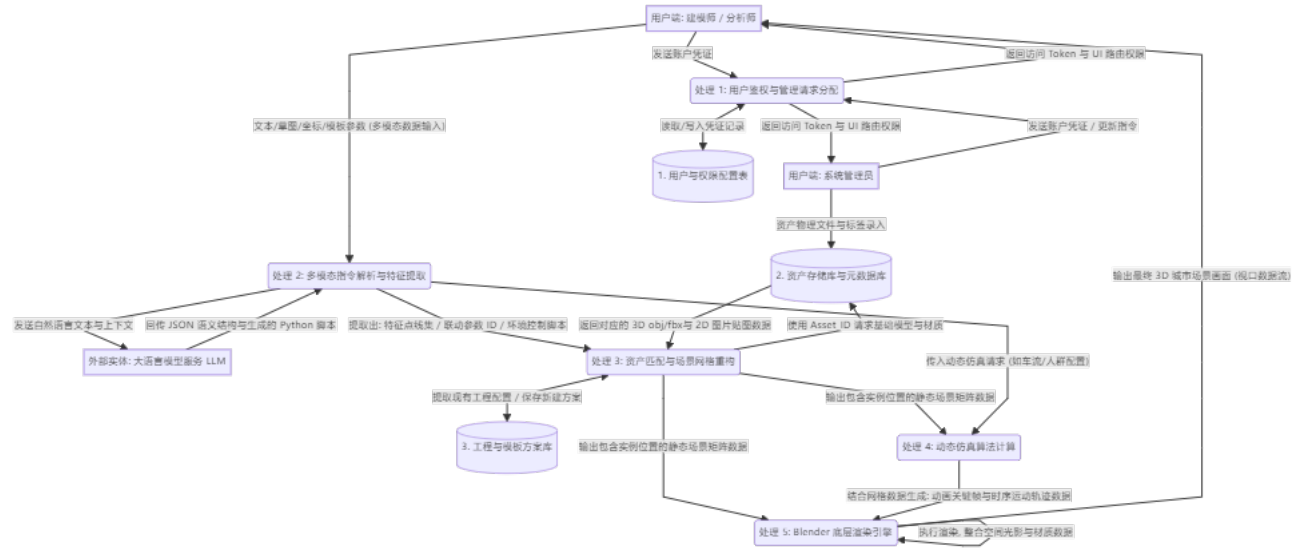


图 4.7: 第一层数据流图 (1 层 DFD)

4.3 数据字典

数据字典是对数据流图 (DFD) 中出现的所有数据元素、数据结构、数据存储和数据流的严格定义与详细说明。本节遵循前面梳理的数据流图与数据需求, 对系统中最核心的数据存储表与数据流进行规范化描述, 为后续的数据库设计与接口开发提供依据。

4.3.1 数据存储字典

本部分主要定义系统底层的三个核心实体数据表结构: 用户登录与权限、资产元数据、以及工程与模板方案。

1. 用户与权限配置表 (User_Account_Table)

- **描述:** 存储系统各类用户 (场景建模师、行业分析师、系统管理员) 的基本信息及其对应的权限角色设定。
- **主体结构:**

字段名称	字段代码	数据类型	长度	必填	默认值	字段说明
用户唯一标识	User_ID	String(UUID)	32	✓	无	系统自动生成的全局唯一用户 ID（主键）
登录账号	Username	String	64	✓	无	用户登录名，支持邮箱或工号
密码哈希值	Password_Hash	String	128	✓	无	经过加密算法处理的密码凭证
用户角色	Role_Type	Enum	16	✓	无	枚举值：Modeler, Analyst, Admin
注册时间	Create_Time	DateTime	-	✓	当前时间	账户在系统中的注册时间
状态	Status	Integer	1	✓	1	1 正常, 0 冻结/封禁

表 1: 用户与权限配置表

2. 资产元数据库表 (Asset_Metadata_Table)

- 描述：记录所有用于城市生成的底层 2D/3D 数字资产的特征标签与物理存储位置。
- 主体结构：

字段名称	字段代码	数据类型	长度	必填	默认值	字段说明
资产唯一标识	Asset_ID	String(UUID)	32	✓	无	用于 Blender 插件调用的唯一资源标记
资产名称	Asset_Name	String	128	✓	无	资源的直观名称（如“常绿乔木_01”）
资产类别	Category	Enum	32	✓	无	诸如 Building, Plant 等枚举值
文件格式	File_Format	String	16	✓	无	如 .blend, .fbx, .obj 等
物理存储路径	Storage_URL	String	256	✓	无	映射到云端或本地存储的绝对路径
默认缩放比	Default_Scale	Float	-	✓	1.0	模型导入场景时的初始化缩放系数
碰撞边界盒	Bounding_Box	JSON	-	×	Null	记录模型在空间中的极值用于碰撞检测

表 2: 资产元数据库表

3. 工程与模板方案表 (Project_Template_Table)

- **描述**：记录场景建模师或行业分析师保存的城市场景工程节点数据，以及预设的联动资产搭配模板。
- **主体结构**：

字段名称	字段代码	数据类型	长度	必填	默认值	字段说明
工程对象标识	Project_ID	String(UUID)	32	✓	无	工程或模板的唯一 ID（主键）
所属用户	Creator_ID	String(UUID)	32	✓	无	关联至 User_ID（外键）
方案名称	Project_Name	String	128	✓	新建工程	工程或模板的显示名称
资产联动配置	Asset_Links	JSON	-	✓	{}	包括路网绑定的特定树木与路灯 ID 映射
环境配置参数	Env_Params	JSON	-	✓	{}	记录当前光照、时间、天气的参数
空间布局矩阵	Layout_Matrix	Text/Blob	-	✓	Null	序列化后的城市网格点线集与坐标矩阵

表 3: 工程与模板方案表

4.3.2 核心数据流字典

本部分主要定义系统中各模块间交互时传递的数据流结构。它反映了系统组件之间是如何进行信息交换的，这些数据流动于内存和接口间，不需要永久保存在静态的数据库表中。

1. 多模态交互数据 (Multimodal_Instruction_Flow)

- **描述**：前端界面收集到用户输入后，传递给主系统、大模型及场景生成模块的数据信息。
- **数据流向**：前端交互界面 -> 主系统 -> 大语言模型 (LLM) -> 场景生成模块
- **主体结构**：

字段名称	字段代码	数据类型	字段说明
会话 ID	Session_ID	String	当前交互操作的全局唯一标识
输入类型	Input_Type	Enum	用户输入形式: Text, Sketch, Coords
原始输入数据	Raw_Data	Blob/Text	用户实际输入的文字、图片或坐标的具体内容
解析后意图	Parsed_Intent	JSON	大模型分析得出的用户操作意图
生成执行脚本	Script	Text	用于控制 Blender 建模的 Python 脚本代码

表 4: 多模态交互数据结构表

2. 动态仿真数据 (Dynamic_Simulation_Stream)

- **描述：** 仿真模块计算出车辆或人群的运动路径后，传递给渲染引擎的运动状态数据。
- **数据流向：** 动态仿真模块 -> Blender 底层渲染引擎
- **主体结构：**

字段名称	字段代码	数据类型	字段说明
帧序号	Frame_Index	Integer	当前动画帧的编号
对象 ID	Entity_ID	String	场景中具体独立模型（汽车、人物等）的标识
空间坐标	Location	Array	对象在场景中的三维位置坐标 [X, Y, Z]
旋转角度	Rotation	Array	对象当前的朝向和旋转角度 [Yaw, Pitch, Roll]
移动速度	Speed	Float	对象的当前移动速度，用于处理加速、减速及避障

表 5: 动态仿真数据结构表

5 功能需求

5.1 功能划分

根据我们的业务流程和系统目标分析，我们将“智能城市生成系统”的总体功能划分为五个核心模块，每个主模块下再细分出若干个子功能组件，以确保系统的功能边界清晰，并为后续模块化开发与系统测试提供明确依据。

下图为智能城市生成系统的功能层级划分结构图：

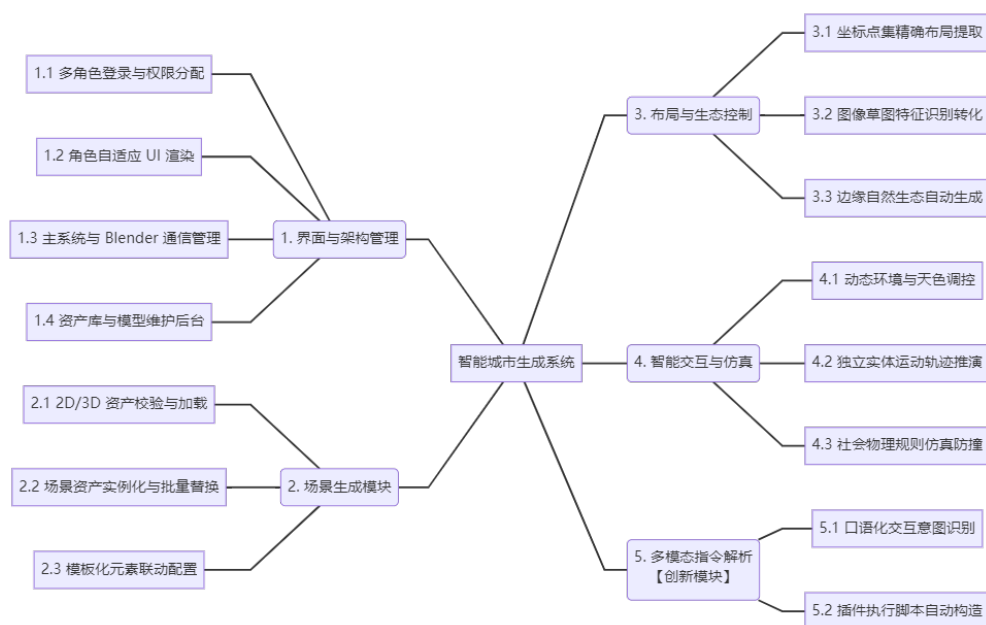


图 5.8: 功能层级划分结构图

模块整体功能说明：

- 1. 界面与架构管理模块：**负责系统账号的安全准入、权限分发、针对不同角色的差异化 UI 渲染，以及建立前端系统与底层 Blender 渲染平台之间的通信机制。
- 2. 场景生成模块：**对接底层的 2D/3D 资产库，实现资产的解析与实例化，以及基于模板组合的街道元素联动配置与批量替换。
- 3. 布局与生态控制模块：**负责城市整体结构与环境排布，支持通过坐标及点集或手绘草图构建和修改核心路网节点，并在城市周边区域自动生成山峦、水系等自然场景。
- 4. 智能交互与仿真模块：**支持通过自然语言交互实现场景时间和天气的实时调控，并在静态路网上结合交通规则生成人群与车辆的动态行驶轨迹。
- 5. 多模态指令解析模块（创新模块）：**负责对接外部大语言模型（LLM），将用户的自然语言意图或手绘草图，转化为底层可直接调用的插件操作与生成脚本。

5.2 功能描述

本节针对上一节“功能划分”中定义的 5 个核心模块，逐一规定其功能描述、输入数据、处理逻辑与输出结果，作为底层代码编写与系统测试的依据。

5.2.1 界面与架构管理模块

1.1 多角色登录与权限分配

- 功能描述：**提供系统的安全入口，验证用户身份并分配对应的操作权限。
- 输入：**用户的登录账号与密码凭证。
- 处理：**在数据库中校验凭证有效性，检索该用户的角色类型（建模师、分析师、管理员）。

- **输出：**登录状态结果，以及携带权限信息的会话 Token。

1.2 角色自适应 UI 渲染

- **功能描述：**根据用户角色动态加载对应的前端界面组件。
- **输入：**用户的权限 Token 与角色标识。
- **处理：**根据权限表路由拦截，隐藏或开放特定菜单（如为分析师隐藏底层 3D 节点面板，为管理员开放资产后台）。
- **输出：**符合特定角色业务诉求的用户工作台界面。

1.3 主系统与 Blender 通信管理

- **功能描述：**建立并维护主系统前端与后台 Blender 渲染进程之间的双向通信。
- **输入：**前端发起的连接请求、存活心跳数据。
- **处理：**通过 WebSocket 或本地端口握手建立连接，并定时监听 Blender 插件进程的存活状态。
- **输出：**底层的通信信道状态，以及连接中断时的异常警报。

1.4 资产库与模型维护后台

- **功能描述：**支持管理员对 2D/3D 数字资产进行统一的上传、分类、查询与数据维护。
- **输入：**资产物理文件（.obj、.fbx、.png 等）、资产名称、分类标签等元数据。
- **处理：**验证文件格式有效性，提取预设缩放与边界数据，将其存储至服务器并更新数据库目录。
- **输出：**资源入库成功的反馈提示，更新后的资源列表。

5.2.2 场景生成模块

2.1 2D/3D 资产校验与加载

- **功能描述：**在执行场景构建前，校验被调用资产的完整性并读取至内存。
- **输入：**目标资产的唯一标识（Asset_ID）。
- **处理：**数据库检索资产路径，检验本地缓存文件是否存在且未损坏，加载 3D 网格与 2D 材质。
- **输出：**准备就绪的资产二进制底层数据。

2.2 场景资产实例化与批量替换

- **功能描述：**将选定的资产在指定的场景坐标中实体化生成，或批量替换已有的组件模型。
- **输入：**目标资产数据、场景空间内的三维坐标点阵集，以及被替换对象的标识。
- **处理：**通过 Blender 底层几何节点运算在目标位置布置新资产，并销毁或隐藏指定的旧资产。
- **输出：**执行替换后的 3D 场景网格结构图。

2.3 模板化元素联动配置

- **功能描述：**通过选择预设的参数模板，一次性自动配置并替换环境中的多种关联元素（如统一配置符合“欧洲古典风”的道路、树木与座椅）。
- **输入：**用户选定的方案模板 ID。
- **处理：**解析模板包含的组合键值对，拆解为一组相关的资产替换指令并依次下发核心引擎。
- **输出：**场景中多类元素的整体联动替换反馈与视觉效果。

5.2.3 布局与生态控制模块

3.1 坐标点集精确布局提取

- **功能描述：**接收用户手动录入的精确空间组参数，构建底层的核心路网节点。
- **输入：**城市核心节点的长度、宽度、方向与间距等 (X,Y) 绝对坐标集。
- **处理：**清洗输入数据的合法性，计算节点间连线并将其转化为 Blender 空间的三维空间网格基准。
- **输出：**标准化的空间布局矩阵数据。

3.2 图像草图特征识别转化

- **功能描述：**利用图像识别技术，从用户上传的二维手绘城市规划图中提取重构所需的点线面数据。
- **输入：**用户上传的 2D 草稿图片位图数据。
- **处理：**通过边缘检测等算法过滤杂色，识别道路主轴与区块边界，并将其映射转换为空间坐标系参数。
- **输出：**从图片提取并转换而成的空间规划布局矩阵。

3.3 边缘自然生态自动生成

- **功能描述：**在已经构建完毕的城市主区域周边空白外围，利用程序化算法生成填充底景。
- **输入：**当前城市场景的边界外框范围数据 (Bounding Box 连线)。
- **处理：**分析外围空白区域的地势高度，自动散布算法生成山峦拓扑结构、湖泊水系以及动态点缀物（如游船）。
- **输出：**完整的城郊生态自然背景模型体系。

5.2.4 智能交互与仿真模块

4.1 动态环境与天色调控

- **功能描述：**通过交互指令实时变更系统的世界环境光设置或粒子特效。
- **输入：**解析后的环境参数指令（如目标时间：“18:00”，目标天气：“大雨”）。
- **处理：**调整 Blender 环境背景图 (HDRI) 的亮度与色温，启用并配置相应的粒子发射器。
- **输出：**附带天气与日照变化的实时场景渲染画面。

4.2 独立实体运动轨迹推演

- **功能描述：**为引入场景的车辆或人群等独立实体分配符合物理现实的运行初态与路径。
- **输入：**生成实体的指令、目标实体的数量、关联的道路网格拓扑地图。
- **处理：**在路网节点中随机或定向分配出生点，基于寻路算法计算每个实体在时间轴上的行进路线。
- **输出：**每个独立实体的分段位移数据轨迹。

4.3 社会物理规则仿真防撞

- **功能描述：**在多实体同时运动时，介入约束判定以避免模型穿模或违背常理运动。
- **输入：**实体的体积碰撞盒参数、实时运动速度、多对象间的相对距离。
- **处理：**执行实时碰撞侦测。当距离到达阈值时，自动调整对象运动状态（如车辆遇行人减速、红灯停止等）。
- **输出：**修正后的连续动画关键帧序列数据。

5.2.5 多模态指令解析模块（创新模块）

5.1 口语化交互意图识别

- **功能描述：**捕捉用户的自然语言输入并利用大模型判定核心控制意图。
- **输入：**用户的原生自然语言文本字符串。
- **处理：**将文本与当前场景状态上下文打包发送至 LLM 接口，经推理识别其业务意图分类（新增/替换/环境调整/仿真控制等）。
- **输出：**格式化、结构化的意图特征 JSON 数据结构。

5.2 插件执行脚本自动构造

- **功能描述：**将识别出的抽象意图精准翻译并转化为底层 Blender 平台可理解、可直接运行的 Python 应用程序接口调用。
- **输入：**格式化的意图特征 JSON 参数。
- **处理：**利用事先注入到 LLM Prompt 中的接口映射规则集，拼接并输出包含指定 Asset_ID 与空间坐标的自动化生成脚本。
- **输出：**可被 Blender 直接安全执行的 `.py` 脚本代码对象。

6 性能与非功能需求

6.1 准确性

系统在处理数据与执行命令时应具备高度的准确性。主要体现在：

- **语义解析准确性：**系统对自然语言命令的意图识别应保持高准确率，确保生成的插件脚本逻辑与用户输入的环境调整或场景生成要求相符。尤其在处理模糊指令（如“随便加点树”）时，须能根据上下文推断出最合理的默认参数配置，不引发底层函数报错。
- **坐标与布局精确度：**从二维图像提取的特征点线或用户手动输入的坐标，在转换为 Blender 三维空间绝对坐标时必须保持精确，保证无形变或尺寸比例错误。此外，在进行建筑网格叠加与替换时，需确保几何拓扑结构的闭合，不允许出现漏面等解析失败现象。
- **仿真防撞严谨性：**在计算人群与车辆的动态推演时，实体的碰撞检测体积判定需与模型真实边缘一致，避免出现穿模或物理违和的位移现象。同时，交通推演必须准确判定信号灯与路口边界流向，确保运动逻辑符合现实常理。

6.2 及时性

考虑到 3D 场景生成与大模型运算的特殊性，响应时间的及时性需分级设定：

- **常规逻辑响应：**平台页面加载、视图切换和表单提交的系统响应时间应控制在 1 秒以内，保证高频交互（如点击菜单、切换资产分类）的顺畅感，不产生可感知的视觉卡顿。
- **大模型接口响应：**自然语言指令提交后，与大语言模型（LLM）的网络通信及语义解析返回时间应控制在合理范围内（建议 3-5 秒左右）。系统应支持状态反馈，在解析过程中向用户展示“思考中”等加载动效，缓解用户的等待焦虑。
- **场景生成响应：**对于简单模板的资产替换应在 10 秒左右完成；对于耗时较长的大规模网格生成与物理仿真，系统必须提供明确的百分比进度提示与后台异步执行机制，防止界面卡死；渲染完毕后，视口的重绘延迟需控制在极低水平以保障体验。

6.3 可扩充性

系统的架构设计需具备良好的横向扩展能力：

- **底层资产扩充：**系统支持标准的资产格式（如.obj, .fbx），管理员可以在不修改系统源码的情况下，通过后台随时上传并上架新的模型与材质，扩充场景表现力。同时，数据库需支持标签化管理，便于随着资产库的日益庞大进行灵活的分组与快速检索。
- **模型接口切换：**大语言模型服务模块应设计为松耦合架构。若未来出现推理更快、能力更强的 LLM 引擎，系统可以只需更改 API 地址与密钥配置即可完成外部模型的平滑切换。系统内部还应预留插件接口，便于日后无缝接入更高级的图像识别算法或自研中间件。

6.4 易用性

面对不同技术背景的用户，系统需建立低门槛的交互体验：

- **角色自适应：**针对无建模基础的行业分析师，系统应隐藏复杂的底层节点与坐标参数面板，核心依赖极简的自然语言输入与模板选择即可完成工作。而对于专业建模师，还可以保留快捷键支持与底层调配入口，兼顾深度编辑的需求。
- **容错与引导提示：**当用户输入了无法解析的模糊语言指令，或调用了本地不存在的资产包时，系统必须能拦截底层异常，并返回清晰、友好的中文提示来指导用户修正操作。界面设计上需遵循直觉原则，常用功能置于视觉焦点区域，降低用户的学习成本。

6.5 易维护性

为保障系统在长期演进中的稳定性，代码设计需符合易维护原则：

- **模块化解耦**：系统的前端 UI、后台管理、LLM 交互中间件以及 Blender 插件底层应当剥离。单一核心的更新（如适配新版本 Blender 的 Python API 变动）不应引发全系统重构。各微服务可通过约定的标准接口进行通信，方便单独进行单元测试。
- **报错日志处理**：系统应在后台记录所有接口调用日志、资源加载记录与异常追踪堆栈，能够协助维护人员快速定位模型解析失败或资产缺损的具体原因。同时，系统应建立定期的平台配置自动备份机制，防止因意外宕机导致的环境配置丢失。

6.6 标准性与先进性

- **标准性**：系统各模块的数据通信一律采用标准的 JSON 格式或 RESTful API 规范；所有接入的 3D 资源要求遵循统一的坐标原点与中心缩放标准；代码规范需遵循 Python PEP8 行业标准。系统的接口文档（如 Swagger）也需与代码保持同步更新以方便后续介入开发。
- **先进性**：本系统改变了单纯依赖繁琐鼠标操作的传统城市建模模式，将当今前沿的多模态大模型技术实质性地引入三维内容生成（AIGC）管线，在系统人机交互模式上具备领先性。这不仅大幅缩短了从构思到落地的设计周期，更为探索全自动“AI 生成世界”验证了切实可行的技术路径。

7 系统运行要求

7.1 硬件配置要求

由于本系统不仅涉及常规的管理与路由逻辑，更需要应对大规模 3D 城市场景的实时渲染、动态交通物理仿真以及部分本地 AI 算法计算，因此对计算机的硬件算力（特别是显卡显存和运行内存）有较高门槛。

1. 服务器端 / 渲染与计算主节点

该节点主要在后台承载 Blender 底层引擎运算、多模态特征转化以及所有的场景物理生成逻辑。

- **CPU（处理器）**：推荐采用 16 核心或以上的商用/工作站级处理器（如 Intel Core i9 或 AMD Ryzen Threadripper 系列）。大规模城市级路网生成的底层布尔运算以及多实体的寻路防撞推演非常依赖多核并行处理能力。
- **GPU（显卡）**：核心配置。最低要求具备 16GB 显存的高性能独立显卡（如 NVIDIA RTX 3080/4080 级别）；推荐配置 24GB 及以上显存的顶级消费卡（如 RTX 4090）或相关专业显卡，以保障同屏加载海量高清材质与实例化植物模型时的渲染帧率，并为本地图像识别与大模型特性提供充足的 CUDA 算力支持。
- **RAM（运行内存）**：系统内存的最低要求为 64GB，推荐 128GB 及以上。多模态场景在替换和撤销操作时，大批量的节点数据会瞬间占据极大的物理内存空间。
- **存储介质（硬盘）**：强烈依赖极速的 I/O 吞吐。推荐配置不低于 2TB 容量、读写速度在 5000MB/s 以上的高速 NVMe M.2 固态硬盘（SSD），从而彻底消除频繁调度数百兆高精度模型资产时的读写瓶颈。

2. 最终用户终端要求

- **场景建模师终端**：作为专业建模人员，若需要在本地直接进行模型网格的细致编辑与修改，推荐使用双路或多路高色域显示器以便对照设计图纸与渲染场景；主机建议为 8 核 CPU、32GB 内存与主流独立显卡（如 RTX 3060 以上）的图形工作站。
- **分析师与管理员终端**：因大量重度渲染与生成过程均已卸载至主节点，这类用户主要基于前端 Web 或轻量化 UI 操作。对硬件依赖较低，市面常规的办公级电脑（如 Intel i5 级别处理器，16GB 内存，集成显卡）配合流畅局域网或宽带网络即可顺畅使用。

7.2 软件配置要求

为保障系统各项功能的正常运行与底层接口的有效通信，渲染主节点与用户终端的软件配置需满足以下要求：

1. 操作系统要求

- **渲染与计算主节点**：推荐使用 Ubuntu 22.04 LTS 或 Windows 11 专业版/Windows Server 2022。该类系统对主流的深度学习框架与高端显卡驱动具有最佳的稳定性与兼容性。
- **最终用户终端**：支持 Windows 10/11、macOS 12 及以上版本。行业分析师端由于主要依赖 Web 端操作，对操作系统的要求可适当放宽。

2. 核心依赖环境

- **3D 建模引擎**：系统核心依赖 **Blender 3.6 LTS** 或更高版本。该版本提供了官方长期支持，且内置了完善的 Geometry Nodes（几何节点）与 Python API (bpy)，是执行批量资产替换与场景静态网格重构的底层基础。
- **开发语言环境**：主系统后端与底层插件均基于 **Python 3.10** 及以上版本开发，需配置对应的依赖包管理工具（如 pip 或 conda）以维系环境一致性。

3. AI 推理与算法组件

- **大语言模型 (LLM) 接口**：系统需能够稳定连通符合设定的外部大模型 API（如 OpenAI 接口、国内主流大模型接口）或本地私有化部署的大模型服务，并要求支持标准的 RESTful API 与 JSON 数据交互格式。
- **计算机视觉框架**：用于处理布局草图识别与特征提取的节点，需预装 OpenCV 视觉库以及主流的深度学习运行框架（如 PyTorch）。

4. 数据库与中间件

- **数据库引擎**：采用 PostgreSQL 14 或 MySQL 8.0 及以上版本存储用户凭证与工程元数据。针对海量的 3D/2D 资产物理文件，推荐对接本地分布式文件系统或云端标准 OSS 对象存储服务。
- **通信与缓存中间件**：为了维系高频的多模态指令流与视图画面回传，系统需使用 Redis 提供高速缓存，并严格依赖 WebSocket 协议栈来实现前后端与 Blender 进程间的双向实时通信。

5. 前端运行环境

- **浏览器要求：**无论是系统管理后台还是行业分析师的交互视图，均要求用户使用最新稳定版的 Google Chrome、Microsoft Edge 或 Safari 等现代浏览器。浏览器需完整支持 HTML5 与 WebGL 标准，以保障动态渲染画面的正常预览。

8 总结

本需求分析报告围绕“智能城市生成系统”的开发目标，梳理了系统的业务逻辑、数据走向、功能模块以及非功能需求。报告具体规划了界面与架构管理、场景生成、布局与生态控制、智能交互与仿真四大基础模块，以解决传统城市建模过程中人工操作繁琐和效率低下的问题。

同时，结合人工智能技术的发展，系统引入了大语言模型（LLM）来实现多模态指令解析的核心功能。通过将自然语言处理与 3D 建模技术相结合，系统实现了基于自然语言的场景构建与环境控制，降低了相关人员的操作门槛。本文档明确了系统的各项业务边界与性能指标，为后续的系统架构设计、代码编写以及软件测试提供了直接的参考依据。